Code-Layout, Readability and Reusability

Template: Code-Layout, Readability and Reusability Evaluation	**Maximum Marks: 5 Marks**
Date: 15 March 2026	**Team ID:** FV-AI-2026-X1
Project Name: FoodVerse AI	**Track:** AI/ML & Generative AI

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description & Implementation in FoodVerse AI	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Strict 4-space indentation across all Python backend routers; 2-space indentation in React JSX components.	Yes	PEP 8 compliant throughout.
2	Proper File Structure	Modular separation of backend/routers, frontend/src, test suites, and documentation folders.	Yes	Clear separation of concerns.
3	Meaningful Variable Names	Self-documenting variable naming (e.g., discoloration_index, target_calories, compliance_score).	Yes	Zero cryptic single-letter variables.
4	Function / Module Documentation	Comprehensive docstrings on every endpoint, computer vision filter, and helper utility.	Yes	Full OpenAPI documentation generated.
5	Type Hinting & Data Schemas	Pydantic v2 schemas (BaseModel) enforcing strict validation on all request and response payloads.	Yes	Prevents runtime malformed payloads.
6	Error Handling & Fallbacks	Try-finally blocks guaranteeing temp file deletion; robust deterministic fallbacks if API is offline.	Yes	Zero unhandled crashes on bad inputs.
7	Security & Secret Isolation	Sensitive API keys managed exclusively via .env and excluded from git commits via .gitignore.	Yes	Zero credentials committed to GitHub.

Modularity & Code Reusability Principles:

• **Modular Routers:** Each of the 5 functional domains lives in its own dedicated router file inside `backend/routers/` allowing independent unit testing.
• **Centralized AI Helper:** Image dimension normalization and audio transcription logic are encapsulated into reusable utility functions.
• **Shared Frontend Components:** The React workbench utilizes shared UI components (stat counters, dropzones, JSON telemetry cards) for unified look and feel.